

Linguaggi di Programmazione e Modelli Computazionali

Esame del 4 Febbraio 2013

Esercizio 1:

Si mostri un automa a stati finiti deterministico con il numero minimo di stati che riconosce la seguente espressione regolare: $((a+(ab))b)^*$

Esercizio 2:

Progettare un automa a stati finiti deterministico che riconosce il linguaggio sull'alfabeto binario $\{0,1\}$ in cui ogni coppia di simboli è diversa dalla precedente. Ad esempio la stringa **0110110** appartiene a tale linguaggio, mentre **0110100** non vi appartiene.

Esercizio 3:

Mostrare una grammatica libera che generi il complemento del linguaggio $\{a^n b^n \mid n \geq 0\}$

Esercizio 4:

Si consideri il linguaggio sull'alfabeto binario $\{0,1\}$ composto dalle stringhe in cui in ogni suffisso il numero di **0** è strettamente inferiore rispetto al numero di **1**. Ad esempio, la stringa **101011** appartiene al linguaggio mentre **1001011** non vi appartiene (a causa del suffisso **001011**). Classificare il linguaggio dicendo se è regolare, libero, ricorsivo, ricorsivamente enumerabile o nemmeno ricorsivamente enumerabile.

Esercizio 5:

Si consideri la grammatica:

$$E \rightarrow E + F \mid F$$

$$F \rightarrow F * T \mid T$$

$$T \rightarrow (E) \mid x$$

dove E, F e T sono variabili e gli altri sono simboli terminali. Trasformare la grammatica eliminando la ricorsione a sinistra e mostrare la relativa tabella di parsing LL(1).

Esercizio 6:

Si consideri la grammatica:

$$F \rightarrow F * T \mid T$$

$$T \rightarrow (F) \mid x$$

Mostrare la tabella di parsing LALR(1).

Esercizio 7:

Descrivere l'algoritmo CYK.

Esercizio 8:

Riportare la dimostrazione del fatto che il linguaggio universale non è ricorsivo.

LPeMC

20130204

January 11, 2015

1 Es1

Hint: $((a+(ab))b)^* = ((a+ab)b)^* = (ab + abb)^*$

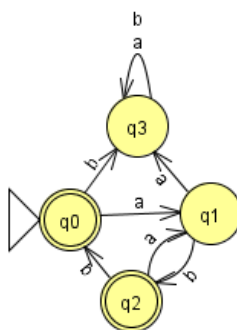


Figure 1: Es1

2 Es2

- *000* NO
- *111* NO
- *101* OK
- *0110110* OK
- *0110100* NO

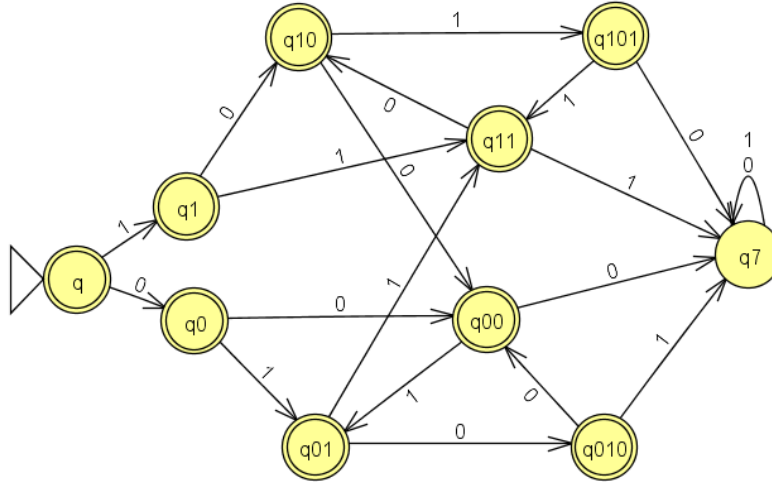


Figure 2: Es2

3 Es3

Hint: Il linguaggio è CFL, ma i CFL non sono chiusi rispetto al complemento. Questo non significa per forza che il complemento di L non possa essere CFL. Ragionando per esaurimento dei casi:

1. $(a+b)^* b (a+b)^* a (a+b)^*$ OK (qualsiasi stringa in cui dopo una b appaia almeno una a);
2. $b (a+b)^*$ OK (qualsiasi stringa che inizia per b)
3. ϵ NO
4. $a^n b^m$ tale che $n \neq m$ OK

$S \rightarrow A b A a A$ (qualsiasi stringa in cui dopo una b appaia almeno una a)

$S \rightarrow b A$ (qualsiasi stringa che inizia per b)

$S \rightarrow a B R$ ($a^n b^m$, $n > m$)

$S \rightarrow R b C$ ($a^n b^m$, $n < m$)

$R \rightarrow a R b \mid \epsilon$

$A \rightarrow BA \mid CA \mid \epsilon$ (epsilon)

$B \rightarrow aB \mid \epsilon$ (qualsiasi sequenza di a, a^*)

$C \rightarrow bC \mid \epsilon$ (qualsiasi sequenza di b, b^*)

4 Es4

Classificare $L = \{ w \mid \text{stringhe nel cui suffisso } \# 0 < \# 1 \}$. La stringa $0^n 1^{n+1}$ appartiene al linguaggio, quindi posso sicuramente escludere che sia regolare (vedi pumping lemma per regolari).

Hint: a quanto pare sembra il solito problema di contare che all'interno di una stringa $\# 1 > \# 0$, linguaggio riconoscibile tramite PDA (quindi è CFL). Il problema è la lettura dell'input: per controllare il suffisso devo leggere la stringa dalla fine all'inizio. Per HP considero il linguaggio CFL, tali linguaggi sono chiusi rispetto al reverse. Considero quindi il linguaggio delle stringhe nel cui prefisso $\# 0 < \# 1$ e provo a costruire il PDA che le riconosca.

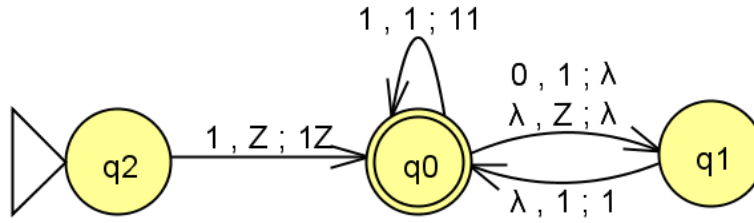


Figure 3: PDA che riconosce stringhe nel cui prefisso $\# 1 > \# 0$

Posso costruire il PDA, quindi $\text{reverse}(L)$ è CFL, quindi L è CFL. Ora tramite pumping lemma dimostro che L non sia regolare. Assumo per HP che L sia regolare, allora esiste n tale che $z = vwx$ con le seguenti proprietà:

1. $|vw| \leq n$
 2. $|w| \geq 1$
 3. $z = vw^i x \in L$ per ogni i
1. Considero $z = 0^n 1^{n+1}$
 2. w contiene (**in ogni caso**) almeno uno 0, quindi per ogni $i > 1$ il $\# 0 > \# 1$, in questo modo esiste almeno un suffisso della stringa con $\# 0 > \# 1$. z non appartiene a L , il PL fallisce, quindi il linguaggio non è regolare.

5 Es5

$E \rightarrow E + F \mid F$
 $F \rightarrow F * T \mid T$
 $T \rightarrow (E) \mid x$

5.1 Pulizia grammatica

C'è ricorsione sinistra diretta (ma non indiretta), modifico la grammatica.

$E \rightarrow F A$
 $A \rightarrow + F A \mid e$ (epsilon)
 $F \rightarrow T B$
 $B \rightarrow * T B \mid e$ (epsilon)
 $T \rightarrow (E) \mid x$

5.2 Calcolo first e follow

	First	Follow
E	First(F) (x	\$)
A	+ e	Follow(E) \$)
F	First(T) (x	First(A) + Follow(A) \$)
B	* e	Follow(F) + \$)
T	(x	First(B) (Follow(B) + \$)

5.3 Costruzione tabella parsing

	(	)	x	+	*	\$
E	F A		F A			
A		e		+ F A		e
F	T B		T B			
B		e		e	* T B	e
T	(E)		x			

Nella tabella di parsing non ci sono conflitti, la grammatica è di tipo LL(1).

6 Es6

$F \rightarrow F * T \mid T$
 $T \rightarrow (F) \mid x$

6.1 Calcolo first e follow

	First	Follow
F	First(T) (x	\$ *)
T	(x	Follow(F) \$ *)

6.2 Costruzione automa

S0

F go to S1
 T go to S2
 (go to S3
 x go to S4

$S' \rightarrow -F$ \$
 $F \rightarrow -F * T$ \$
 $F \rightarrow -T$ \$
 $T \rightarrow -(F)$ \$
 $T \rightarrow -x$ \$
 $F \rightarrow -F * T$ *
 $F \rightarrow -T$ *
 $T \rightarrow -(F)$ *
 $T \rightarrow -x$ *

Da cui raggruppando si ottiene

$S' \rightarrow -F$ \$
 $F \rightarrow -F * T$ \$*
 $F \rightarrow -T$ \$*
 $T \rightarrow -(F)$ \$*
 $T \rightarrow -x$ \$*

S1

* go to S5
 $S' \rightarrow F-$ \$
 $F \rightarrow F -* T$ \$*

S2

$F \rightarrow T-$ \$*

S3

F go to S6
 T go to S7
 (go to S8
 x go to S9
 $T \rightarrow (-F)$ \$*
 $F \rightarrow -F * T$)*
 $F \rightarrow -T$)*
 $T \rightarrow -(F)$)*
 $T \rightarrow -x$)*

S4

$T \rightarrow x-$ \$*

S5
T go to S10
(go to S3
x go go S4
 $F \rightarrow F * \neg T$ \$*
 $T \rightarrow \neg(F)$ \$*
 $T \rightarrow \neg x$ \$*

S6
) go to S11
T go to S12
 $T \rightarrow (F \neg)$ \$*
 $F \rightarrow F -* T$)*

S7
 $F \rightarrow T-$)*

S8
F go to S13
T go to S7
(go to S8
x go to S9
 $T \rightarrow (\neg F)$)*
 $F \rightarrow \neg F * T$)*
 $F \rightarrow \neg T$)*
 $T \rightarrow \neg(F)$)*
 $T \rightarrow \neg x$)*

S9
 $T \rightarrow x-$)*

S10
 $F \rightarrow F * T-$ \$*

S11
 $T \rightarrow (F)-$ \$*

S12
T go to S14
(go to S8
x go to S9
 $F \rightarrow F * \neg T$)*
 $T \rightarrow \neg(F)$)*
 $T \rightarrow \neg x$)*

S13
) go to S15
* go to S12
 $T \rightarrow (F \neg)$)*
 $F \rightarrow F -* T$)*

S14
 $F \rightarrow F * T-$)*

S15
 $T \rightarrow (F)-$)*

Quello ottenuto è l'automa di un parser LR(1), secondo il criterio LALR(1) posso raggruppare gli stati:

- S15 e S11
- S14 e S10
- S13 e S6
- S12 e S5
- S9 e S4
- S8 e S3
- S7 e S2

Il nuovo automa LALR è:

S0

F go to S1
T go to S2
(go to S3
x go to S4
S' → -F \$
F → -F * T \$*
F → -T \$*
T → -(F) \$*
T → -x \$*

S1

* go to S5
S' → F- \$
F → F -* T \$*

S2

F → T- \$*)

S3

F go to S6
T go to S2
(go to S3
x go to S4
T → (-F) \$*)
F → -F * T \$*)
F → -T \$*)
T → -(F) \$*)
T → -x \$*)

S4

T → x- \$*)

S5

T go to S10
(go to S3
x go to S4
F → F * -T \$*)
T → -(F) \$*)
T → -x \$*)

S6

) go to S11

```

* go to S5
T -> ( F -)      $*)
F -> F -* T      )*)

```

```

S10
F -> F * T-      $*)

```

```

S11
T -> ( F )-      $*)

```

6.3 Tabella di parsing LALR

	(	)	*	x	\$	F	T
0	s3			s4		g1	g2
1			s5		ACC		
2		F -> T	F -> T		F -> T		
3	s3			s4		g6	g2
4		T -> x	T -> x		T -> x		
5	s3			s4			g10
6		s11	s5				
10		F -> F * T	F -> F * T		F -> F * T		
11		T -> (F)	T -> (F)		T -> (F)		

7 Es7

Descrivere CYK.

8 Es8

Dimostrare che Lu non è ricorsivo.